Activity 2 — Web Scraping with Python

What you'll learn

Web scraping means pulling data straight off a web page with code. You'll scrape the **CAR Region Open Data Portal** — a live web page with three tables (weather stations, schools, and tourism sites) — do a **first pass of data cleaning** (handle missing values and duplicates), and build an analysis dashboard.

> **The page we scrape:** `https://datasciencebaguio-production.up.railway.app/datasets/car_data`
> **Honest note:** this portal holds **simulated sample data** made for this workshop. The numbers are realistic but invented — not official government figures. What's real is the **skill**.

Before you start

- A **Google account** to use Google Colab.
- An **internet connection** (we pull the page live — no file to upload).

Run each cell with **Shift + Enter**, top to bottom.

Step 1 — Install the scraping tools

COPY THIS CODE

```
!pip install beautifulsoup4 requests
```

> **You should see:** install messages or "Requirement already satisfied".

Step 2 — Load the libraries

COPY THIS CODE

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display

print("Libraries loaded.")
```

> **You should see:** `Libraries loaded.`

Step 3 — Download the web page

`requests.get` fetches the raw page from the portal. A status of **200** means success.

COPY THIS CODE

```
url = 'https://datasciencebaguio-production.up.railway.app/datasets/car_data'

try:
    response = requests.get(url, timeout=15)
    print(f"Status code: {response.status_code}")
    if response.status_code == 200:
        print("Page downloaded successfully.")
    else:
        print("Got a response, but not 200. The site may be busy.")
except Exception as e:
    print(f"Could not reach the site: {e}")
    print("Check your internet connection and try again.")
```

> **You should see:** `Status code: 200` and "Page downloaded successfully."

Step 4 — Parse the HTML and look at the page

`BeautifulSoup` turns the raw HTML into something we can search. We read the page heading, the notice, and count how many tables are on the page.

COPY THIS CODE

```
soup = BeautifulSoup(response.text, 'html.parser')

print("Page heading:", soup.find('h1').text)

notice = soup.find('p')
if notice:
    print("\nNotice on the page:")
    print(" ", notice.text.strip())

tables_on_page = soup.find_all('table')
print(f"\nThis page has {len(tables_on_page)} tables.")

print("\nSection titles found:")
for h2 in soup.find_all('h2'):
    print(" -", h2.text)
```

You should see: the page heading, the "simulated data" notice, "This page has 3 tables", and the three section titles.

Step 5 — Scrape all three tables at once

`pd.read_html` reads every table on the page into a list. This page has three.

COPY THIS CODE

```
tables = pd.read_html(response.text)
print(f"Scraped {len(tables)} tables from the page.\n")

weather_df = tables[0]
schools_df = tables[1]
tourism_df = tables[2]

print(f"Weather stations: {len(weather_df)} rows")
print(f"Schools: {len(schools_df)} rows")
print(f"Tourism sites: {len(tourism_df)} rows")
```

You should see: Weather **47**, Schools **51**, Tourism **40** rows. (The weather and schools counts look a little high — there's a duplicate row hiding in each. We'll catch it in Step 7.)

Step 6 — Look at what we scraped

Always eyeball your data before analyzing it.

COPY THIS CODE

```
print("WEATHER STATIONS (first 5):")
display(weather_df.head())
print("SCHOOLS (first 5):")
display(schools_df.head())
print("TOURISM SITES (first 5):")
display(tourism_df.head())
```

> **You should see:** the first 5 rows of each table.

Step 7 — Check the data quality

Real scraped data is rarely perfect. Before analyzing, check each table for **missing values** (empty cells) and **duplicate rows**.

COPY THIS CODE

```
for name, df in [('WEATHER', weather_df), ('SCHOOLS', schools_df), ('TOURISM',
tourism_df)]:
    print(f"{name}: {df.isnull().sum().sum()} missing cells, {df.duplicated().sum()}
duplicate rows")

print("\nMissing cells per column (weather):")
print(weather_df.isnull().sum()[weather_df.isnull().sum() > 0])

print("\nMissing cells per column (schools):")
print(schools_df.isnull().sum()[schools_df.isnull().sum() > 0])
```

> **You should see:** Weather and Schools each report some missing cells and **1 duplicate row**; Tourism is clean (0 and 0).

Step 8 — A first pass at cleaning

The tourism table is already clean, but **weather** and **schools** have a few empty cells and a duplicate row. We do a light, sensible clean-up:

- **Remove duplicate rows** (the exact same record appearing twice).
- **Fill missing numbers** with the column **average**.
- **Fill missing text** with the **most common value** (the mode).

> This is just a **first pass**. Full cleaning (outliers, data types, validation) is a bigger topic for later.

COPY THIS CODE

```
# 1) Remove duplicate rows
weather_df = weather_df.drop_duplicates().copy()
schools_df = schools_df.drop_duplicates().copy()

# 2) Fill missing NUMBERS with the column average
for col in ['Humidity_pct', 'Rainfall_mm']:
    weather_df[col] = weather_df[col].fillna(round(weather_df[col].mean()))
schools_df['Teachers'] =
schools_df['Teachers'].fillna(round(schools_df['Teachers'].mean()))
schools_df['Year_Established'] =
schools_df['Year_Established'].fillna(schools_df['Year_Established'].median())

# 3) Fill missing TEXT with the most common value (mode)
weather_df['Condition'] = weather_df['Condition'].fillna(weather_df['Condition'].mode()
[0])
schools_df['Type'] = schools_df['Type'].fillna(schools_df['Type'].mode()[0])

print("After the first cleaning pass:")
for name, df in [('WEATHER', weather_df), ('SCHOOLS', schools_df), ('TOURISM',
tourism_df)]:
    print(f"  {name}: {len(df)} rows, {df.isnull().sum().sum()} missing cells,
{df.duplicated().sum()} duplicates")

print("\nNote: this is a first pass only. Full cleaning comes later.")
```

> **You should see:** after cleaning, Weather back to **46** rows, Schools **50** rows, and **0 missing cells, 0 duplicates** everywhere.

Step 9 — Quick summary numbers

COPY THIS CODE

```
print("WEATHER")
print(f"  Average temperature: {weather_df['Temperature_C'].mean():.1f} C")
print(f"  Average rainfall: {weather_df['Rainfall_mm'].mean():.1f} mm")

print("\nSCHOOLS")
print(f"  Total enrollment: {schools_df['Enrollment'].sum():,} students")
print(f"  Average class ratio: {schools_df['Student_Teacher_Ratio'].mean():.1f} students
per teacher")

print("\nTOURISM")
print(f"  Total annual visitors: {tourism_df['Annual_Visitors'].sum():,}")
print(f"  Average rating: {tourism_df['Rating'].mean():.2f} out of 5")
```

> **You should see:** headline figures for each table (e.g. avg temperature ~24 C).

Step 10 — Correlation between two variables

A **correlation** tells us if two number columns move together. It ranges from **−1 to +1**: near **+1** they rise together, near **−1** one rises as the other falls, near **0** there's no clear relationship. We check **elevation** vs **temperature** in the weather data.

> In real life we'd expect higher places to be cooler (a negative correlation). Our data is randomly simulated, so don't over-read the exact number — the point is learning *how* to measure a relationship.

COPY THIS CODE

```
x = 'Elevation_m'
y = 'Temperature_C'

r = weather_df[x].corr(weather_df[y])
print(f"Correlation between {x} and {y}: {r:.3f}")

if r > 0.5:
    print("That's a fairly strong POSITIVE relationship.")
elif r < -0.5:
    print("That's a fairly strong NEGATIVE relationship.")
else:
    print("That's a weak relationship (close to zero).")

plt.figure(figsize=(8, 5))
plt.scatter(weather_df[x], weather_df[y], alpha=0.6, color='purple')
plt.title(f'{x} vs {y}  (correlation = {r:.2f})', fontweight='bold')
plt.xlabel('Elevation (m)')
plt.ylabel('Temperature (C)')
plt.grid(alpha=0.3)
plt.show()
```

You should see: a correlation value (somewhere around −0.4 on this sample) and a scatter plot of the two variables.

Step 11 — The analysis dashboard

Six charts in one figure, pulling from all three scraped tables.

COPY THIS CODE

```
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle('CAR Region Data Dashboard (Simulated Sample Data)', fontsize=15,
fontweight='bold')

# 1. Temperature by province
weather_df.boxplot(column='Temperature_C', by='Province', ax=axes[0, 0])
axes[0, 0].set_title('Temperature by Province')
axes[0, 0].set_xlabel('')
axes[0, 0].tick_params(axis='x', rotation=45)

# 2. Students by school type
by_type = schools_df.groupby('Type')['Enrollment'].sum()
axes[0, 1].pie(by_type.values, labels=by_type.index, autopct='%1.0f%%', startangle=90)
axes[0, 1].set_title('Students by School Type')

# 3. Rainfall vs elevation
axes[0, 2].scatter(weather_df['Elevation_m'], weather_df['Rainfall_mm'], alpha=0.6,
color='teal')
axes[0, 2].set_title('Rainfall vs Elevation')
axes[0, 2].set_xlabel('Elevation (m)')
axes[0, 2].set_ylabel('Rainfall (mm)')

# 4. Tourism ratings
axes[1, 0].hist(tourism_df['Rating'], bins=12, color='gold', edgecolor='black')
axes[1, 0].axvline(tourism_df['Rating'].mean(), color='red', linestyle='--',
                   label=f"Mean {tourism_df['Rating'].mean():.2f}")
axes[1, 0].set_title('Tourism Site Ratings')
axes[1, 0].set_xlabel('Rating')
axes[1, 0].legend()

# 5. Visitors by tourism type
by_t = tourism_df.groupby('Type')['Annual_Visitors'].sum().sort_values()
axes[1, 1].barh(by_t.index, by_t.values, color='coral')
axes[1, 1].set_title('Annual Visitors by Type')
axes[1, 1].set_xlabel('Visitors')

# 6. Weather conditions
cond = weather_df['Condition'].value_counts()
axes[1, 2].bar(range(len(cond)), cond.values, color='lightblue')
axes[1, 2].set_xticks(range(len(cond)))
axes[1, 2].set_xticklabels(cond.index, rotation=45, ha='right')
```

```
axes[1, 2].set_title('Weather Conditions')
axes[1, 2].set_ylabel('Stations')

plt.tight_layout()
plt.show()
print("Dashboard built from the scraped CAR data.")
```

You should see: a 6-chart dashboard (boxplot, pie, scatter, histogram, bar, bar).

Tips & common errors

- Run cells **top to bottom** with Shift + Enter.
- **Status not 200, or a connection error?** Check your internet and re-run Step 3. The portal may be waking up — try once more.
- **KeyError on a column?** Column names are spelling- and case-sensitive (e.g. `Temperature_C`, `Annual_Visitors`). Use them exactly.
- Always check a site's **terms** before scraping it, and always be clear whether data is **real or simulated**.

What's next

In **Activity 3** you'll take this scraped tourism data and run a full **exploratory data analysis**.
